Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Dersin tarihi 18 Ekim 2021 Pazartesi. Ders Zoom üzerinden yapıldı.
  - Hoca, garanti olsun diye 1. bölümün 29. slaytından, yani çoklu programlamadan geri dönüp başladı.
  - Ders yaklaşık 01:52'de biter.
- Çoklu programlama (multiprogramming) ve zaman paylaşımı (time sharing) tekrarı
  - CPU ani yükü ve G/Ç ani yükü (CPU burst, I/O burst).
  - Tepki süresi (response time), takas (swapping) ve sanal bellek (virtual memory).
- Kesmeyle sürülen işletim sistemi
  - Donanım ve yazılım kesmeleri (trap, exception).
- İkili mod (dual mode): kullanıcı modu (user mode) ve çekirdek modu (kernel mode), mod biti.
- İşletim sisteminin görevleri
  - Süreç yönetimi (process management), bellek yönetimi (memory management), depolama yönetimi (storage management), G/Ç alt sistemi (I/O subsystem).
- Koruma ve güvenlik (protection and security)
  - Kullanıcı kimliği, grup kimliği, yetki yükseltme (privilege escalation).
- Çekirdekteki veri yapıları: bağlı liste, çift yönlü ve dairesel liste, ikili arama ağacı, hash.
- 2. bölüm: İşletim sistemi yapıları
  - İşletim sistemi servisleri.
  - Kullanıcı arayüzleri: komut satırı (CLI), grafik arayüz (GUI), toplu iş (batch).
  - Sistem çağrıları (system calls) ve API'ler. Dosya kopyalama örneği.
  - Sistem çağrısına parametre aktarma yöntemleri.
  - Sistem çağrısı türleri.
- Öğrenci soruları: dosya sistemi türleri arasındaki farklar, grup kimliği (group ID), boru (pipe).

Hocanın Özellikle Vurguladığı Kısımlar

- Slaytların sonuna kadar sorumlusunuz.
  - Hoca 1. bölümde 46. slayttan sonrasını anlatmadı. Bu, o slaytlardan sorumlu olmadığınız anlamına gelmez. Geri dönüp okuyun.
- Kesmenin ne olduğunu ve nasıl çalıştığını anlamak önemlidir.
  - Her sistem çağrısında aslında bir kesme mekanizması kullanılır.
- Bir instruction'ı çalıştıranın işletim sistemi mi kullanıcı mı olduğunu yalnız yazılımla anlayamazsınız.
  - Yazılıma "Sen işletim sistemi misin?" diye sorsanız "Evet" diyebilir.
  - Bu yüzden ikili mod donanım ile yazılımın birlikte desteklediği bir yapıdır.

- `fopen` bir sistem çağrısı değildir, bir kütüphane fonksiyonudur. Altında `open` sistem çağrısını kullanır.
  - Aynı şekilde `printf` bir fonksiyondur. Sistem çağrısı karşılığı `write`'tır.
- Program ile süreç (process) farklıdır.
  - Program pasif bir varlıktır, diskte durur.
  - Süreç, belleğe alınmış ve çalışan aktif bir varlıktır. Sınıf ile nesne arasındaki ilişkiye benzer.
- Bir çocuk sürecin dönüş değerini hiç okumazsanız, işletim sisteminde bir kayıt kalır ve boşuna kaynak tutar. Bu konu süreç yönetiminde tekrar işlenecek.
- Sistem çağrılarının adlarına takılmayın.
  - Windows'ta `CreateProcess`, UNIX'te `fork` gibi adlar farklıdır. Kitap tek tip adlar (open, read, write) kullanır.
- Soru sorun. Soru dersi etkileşimli yapar ve daha çok şey öğrenirsiniz.
  - Yazıyla soru sorarken soruyu açık ve anlaşılır yazın. "pipe?" bir soru değildir.
- Sunucu sistemlerinde genelde grafik arayüz kullanılmaz.
  - Grafik arayüz hataya daha açıktır. Örnek: yanlışlıkla üst seviyedeki bir klasörü silmek.
  - Birçok sistem, yetkili kullanıcının (root) grafik arayüze giriş yapmasını kısıtlar.

Kısa Tekrar Notları

- Sürecin yaşam döngüsü: CPU ani yükü ve G/Ç ani yükü dönüşümlü gelir.
- Çoklu programlama: Bellekte birden fazla süreç tutulur. Biri G/Ç beklerken CPU başka bir süreci çalıştırır.
- Zaman paylaşımı: CPU yoğun kısmın bitmesi beklenmez. Belli zaman aralıklarıyla süreçler arasında geçilir. Amaç tepki süresini düşürmektir.
- Süreç sayısı artınca az kullanılanlar ikincil belleğe atılır (swap out), gerekince geri getirilir (swap in).
- Sanal bellek: Sürecin yalnız bir kısmı fiziksel bellekte olsa da süreç çalışabilir.
- Kesme türleri:
  - Donanım kesmesi: disk, klavye gibi aygıtlardan gelir.
  - Yazılım kesmesi (trap, exception): çalışma hatası (sıfıra bölme), sistem çağrısı veya yetkisiz işlem sonucu oluşur.
- İkili mod:
  - Kullanıcı modu ve çekirdek modu. Hangisinde olunduğunu donanımdaki mod biti gösterir.
  - Kitaptaki örnekte kullanıcı modu = 1, çekirdek modu = 0.
  - Ayrıcalıklı komutlar (privileged instructions; zamanlayıcıyı programlamak, bellek yönetimi komutları gibi) yalnız çekirdek modunda çalışır.
  - Sistem çağrısı: kullanıcı modu, trap ile çekirdek modu (mod biti 0), işi yap, dönmeden mod biti 1, kullanıcı modu.
- Süreç yönetiminin görevleri: süreç oluşturma ve silme, askıya alma ve sürdürme, eşzamanlama (synchronization), süreçler arası iletişim, kilitlenme (deadlock) ile başa çıkma.
- Bellek yönetimi: Hangi sürecin hangi parçasının ne zaman bellekte olacağına karar verir.
- Depolama yönetimi: Aygıtların farklılığını soyutlar (abstraction). Üstüne bir dosya sistemi koyar.
- Yazmaç ve önbellek yönetimi işletim sisteminin işi değildir:
  - Yazmaç kullanımını derleyici belirler.
  - Önbelleği donanım yönetir.
  - Geri kalan seviyeler arasında veri taşımayı işletim sistemi yönetir.
- G/Ç alt sistemi: tamponlama (buffering), önbelleğe alma (caching), sarmalama (spooling) ve tek tip bir aygıt sürücüsü arayüzü.
- UNIX erişim modeli: kullanıcı (user), grup (group), diğerleri (others). Her biri için okuma, yazma, çalıştırma (r, w, x) hakları vardır.
- Sistem çağrısına parametre aktarma:
  1. Yazmaçlarda.
  2. Bellekte bir blok (tablo) içinde. Bloğun adresi bir yazmaçla verilir. Kitaba göre Linux ve Solaris bunu kullanır.
  3. Yığıtta (stack). Program parametreleri yığıta iter (push), işletim sistemi alır (pop).

Detaylı Açıklamalar

Çoklu programlama ve zaman paylaşımı

Bilgisayar sistemindeki her şey bir kaynaktır ve etkin kullanılması gerekir. Bir sürecin çalışması iki tür kısımdan oluşur:

- **CPU ani yükü (CPU burst):** İşlemcinin yoğun kullanıldığı kısım.
- **G/Ç ani yükü (I/O burst):** Bir bilginin yazılmasını veya gelmesini beklediği kısım. Bu sırada işlemci boş kalır.

Çoklu programlama, bu boşluğu doldurmak için bellekte birden fazla program tutar. Bir süreç G/Ç başlatınca CPU boşuna beklemez, bellekteki başka bir süreci çalıştırır.

Bir sürecin CPU yoğun kısmı çok uzunsa, diğer süreçler hiç ilerlemiyormuş gibi görünür. Örnek: Bir programı açmak için tıklarsınız ama uzun süre bir şey olmaz. Fare imleci geriden gelir, bastığınız tuşlar ekrana geç düşer.

Zaman paylaşımı bunu çözer. CPU yoğun kısmın bitmesi beklenmez. Belli zaman aralıklarıyla süreçler arasında geçilir: önce birinden bir parça, sonra diğerinden bir parça çalıştırılır. Böylece tek işlemcide bile birden fazla iş aynı anda yapılıyormuş gibi görünür. Amaç tepki süresini düşürmektir. Geçişlerin nasıl yapılacağı zamanlama (scheduling) konusunda işlenecek.

Zaman paylaşımı için bellekte çok sayıda süreç gerekir. Süreç oluşturma ise bellek dolunca durmaz. Bu yüzden:

- Az kullanılan süreçler ikincil belleğe taşınır ve gerekince geri getirilir. Buna **takas (swapping)** denir.
- Bellek ne kadar büyük olursa olsun, süreç sayısı arttıkça yetmez. **Sanal bellek** sayesinde sürecin yalnız bir kısmı fiziksel bellekte olduğu hâlde süreç çalışabilir.

Slayttaki çoklu programlama bellek düzeninde işletim sistemi en üstte, işler altında durur. İşletim sistemi en altta da olabilir. İşler bitip bellekten çıkınca boşluklar oluşur. Yeni bir iş bu boşluklara sığmayabilir. Bu sorun bellek yönetimi konusunda işlenecek.

Kesmeler

Modern bir işletim sistemi kesme mekanizması olmadan düşünülemez. Kesmeler iki temel türe ayrılır:

- **Donanım kesmesi:** Bir aygıtın ürettiği kesmedir. Disk, klavye gibi karakter aygıtları veya başka bir arayüz kaynak olabilir.
- **Yazılım kesmesi (trap veya exception):**
  - Çalışma zamanında oluşan bir hata. Örnek: sıfıra bölme.
  - İşletim sisteminden servis isteği. Örnek: sistem çağrısı.
  - Bir sürecin başka bir sürecin belleğini değiştirmeye çalışması veya yetkisi olmayan bir komutu çalıştırması. Süreçler birbirinden korunmalıdır.

İkili mod (dual mode)

İşlemcide toplama, çıkarma gibi herkesin çalıştırabileceği komutlar vardır. Bazı komutları ise yalnız işletim sistemi çalıştırmalıdır. Örnekler:

- zamanlayıcıyı (timer) programlamak,
- sayfalama (paging) ile ilgili bellek yönetimi komutları,
- bir bellek bölümünün okunabilir, yazılabilir veya çalıştırılabilir olacağını belirlemek.

Bir komutu kimin çalıştırdığı yalnız yazılımla anlaşılamaz. Yalnız donanımla da çözülemez. İki tarafın da desteklediği ortak bir yapı gerekir.

- İşlemci iki çalışma modunu destekler: **kullanıcı modu** ve **çekirdek modu**.
- Donanımdaki bir **mod biti** hangi modda olunduğunu gösterir. Hangi değerin hangi moda karşılık geldiğine mimari karar verir. Kitaptaki örnekte kullanıcı modu 1, çekirdek modu 0'dır.
- Açılışta BIOS, önyükleyiciyi çekirdek modunda başlatır. Önyükleyici ve işletim sistemi aynı seviyededir.

- Bundan sonra modlar arası geçişi işletim sistemi ve donanım yönetir.

Örnek akış: Bir kullanıcı süreci dosya açmak için `fopen` çağırır.

1. `fopen`, C çalışma zamanı kütüphanesinin (runtime library) sunduğu bir fonksiyondur. Altta `open` sistem çağrısını kullanır.
2. `open` çağrılınca bir trap (kesme) oluşur. Mod biti 0 olur, çekirdek moduna geçilir.
3. Kesme işleyicisi bunun bir sistem çağrısı olduğunu ve kaç numaralı çağrı olduğunu anlar. Örnek: 300 numaralı çağrı.
4. Sistem çağrısı tablosunda bu numaranın karşılığı bulunur ve gereken iş yapılır.
5. Kullanıcı moduna dönmeden önce mod biti tekrar 1 yapılır.

İşletim sisteminin görevleri

Süreç yönetimi. Program, bir programlama dilinde yazılmış ve diskte duran pasif bir varlıktır. Çalışması için belleğe alınması ve komutlarının sırayla çalıştırılması gerekir. Bunu yapan canlı varlığa **süreç** denir. Bu ilişki nesne yönelimli programlamadaki sınıf ile nesne ilişkisine benzer.

İşletim sistemi bir sürecin bütün yaşam döngüsünü izlemelidir. Bunun için veri yapıları gerekir. Bu yapılarda şu bilgiler tutulur:

- sürecin eriştiği G/Ç aygıtları,
- süreci çalıştıran kullanıcı,
- sürecin aldığı CPU zamanı,
- sürecin kaldığı yer (program sayacı).

Zaman paylaşımında süreçler durdurulup sürdürüldüğü için her sürecin kaldığı yeri bilmek gerekir. Süreç çok iş parçacıklı (multi-thread) ise her iş parçacığı için ayrı bilgi tutulur.

Süreç yönetiminin görevleri:

- kullanıcı ve sistem süreçlerini oluşturmak ve silmek,
- süreçleri askıya almak ve sürdürmek,
- süreçler arası eşzamanlama ve iletişim mekanizmaları sağlamak,
- kilitlenme (deadlock) ile başa çıkmak.

Süreci silmek yalnız belleği boşaltmak değildir. Süreç bittiğinde bir dönüş kodu (return code) üretir: başarılı mı, hata mı? İşletim sistemi bu bilgiyi tutar. Ana süreç bu bilgiyi hiç okumazsa, kayıt sistemde kalır ve kaynağı boşuna tutar.

Süreçler kaynaklar için yarışır. Örnek: Bir süreç 1. ve 2. kaynağı tutar, 3. kaynağı bekler. Başka bir süreç 3. kaynağı tutar, 1. ve 2. kaynağı bekler. Kimse elindekini bırakmazsa iki süreç de kilitlenir.

Bellek yönetimi. Bir sürecin çalışması için en azından bir parçasının ve kullandığı verinin bellekte olması gerekir. Hangi parçanın ne zaman bellekte olacağına bellek yönetimi karar verir. Kısa, orta ve uzun vadeli zamanlayıcılar (short, mid, long term scheduler) ileride işlenecek.

Belleğe alınacak işler rastgele seçilirse, hepsi G/Ç yoğun çıkabilir. O zaman süreçler tren gibi önce CPU kuyruğunda, sonra G/Ç kuyruğunda bekler. Bu yüzden hangi işin belleğe alınacağı yönetilmelidir.

Depolama yönetimi. Elimizde farklı ortamlar vardır:

Ortam	Erişim	Kapasite	Hız
Manyetik bant	Sıralı	Çok büyük	Çok düşük
Disk	Sıralı ile rastgele arası	Banttan küçük, bellekten büyük	Banttan hızlı

Her ortamın teknolojisi ve denetleyicisi farklıdır. İşletim sistemi bu farklılıkları soyutlar. Bunun üstüne bir soyutlama daha koyar: **dosya sistemi**. Dosya ve klasörlerle çalışırız, ham veriyi (raw data) doğrudan kullanmayız. Dosya sistemi dosya oluşturma, silme gibi işlemleri herkese tek tip sunar. Her kullanıcının dosyasını da başkalarından korur.

Öğrenci, dosya sistemleri (ext, FAT, günlüklü dosya sistemleri) arasındaki farkı sordu. Hoca, i-düğüm (inode), blok, doğrudan ve dolaylı adresleme konuşulmadan bunun anlatılamayacağını söyledi. Konu dosya sistemleri bölümünde işlenecek.

Depolama hiyerarşisi ve tutarlılık. Diskteki bir veriye ihtiyaç olunca veri önce ana belleğe, sonra önbelleğe, sonra yazmaca gelir.

- Yazmaç kullanımını derleyici belirler. Hangi değerin yazmaçta tutulacağına derleyicinin ürettiği kod karar verir.
- Önbelleği CPU yönetir. Bir adrese erişilince çevresindeki veri de önbelleğe çekilir.
- Diğer seviyeler arasındaki taşımayı işletim sistemi yönetir. Çok erişilen veriye "sıcak", az erişilene "soğuk" veri denir.

Verinin bütün seviyelerde tutarlı kalması gerekir. Tek işlemcide bu kolay yönetilir. Çok işlemcili sistemlerde zordur. Buna **önbellek tutarlılığı (cache coherence)** denir. Konu dağıtık sistemler kısmında işlenecek.

G/Ç alt sistemi. İşletim sistemi donanımın karmaşıklığını kullanıcıdan saklar. Örnek: C'de `printf` ile ekrana yazarken satır sonu koymazsanız veya `fflush` çağırmazsanız çıktı hemen yazılmaz.

- Çalışma zamanı kütüphanesi karakterleri bir tamponda biriktirir, tampon dolunca işletim sistemine gönderir.
- İşletim sistemi de ikinci bir tampon tutabilir.

Bu yüzden karakter karakter yazan bir program ile blok hâlinde yazan bir program arasında beklenen büyük süre farkı görülmez. G/Ç alt sisteminin görevleri:

- **Tamponlama (buffering).**
- **Önbelleğe alma:** Bir dosyanın ilk baytı okununca sonraki baytlar da büyük olasılıkla okunacaktır. Bu yüzden tek karakter yerine bir blok okunur ve önbellekte tutulur.
- **Tek tip aygıt sürücüsü arayüzü** sunma.

Koruma ve güvenlik

İşletim sistemi, sistemi kimin kullandığını bilmeli ve kullanıcılar arasında yetki seviyeleri kurmalıdır.

- Her kullanıcı için ayrı bir kimlik oluşturulmalıdır.
- Kimlik doğrulama (authentication) ve yetkilendirme (authorization) yapılmalıdır.
- Güvenlik yalnız dışarıdan gelen saldırılar için değildir. Saldırı içeriden de gelebilir.

Windows kurulumunda genelde hemen yönetici (administrator) yetkili bir kullanıcı oluşturulur. Tek kullanıcılı kişisel bilgisayarda bu çok sorun olmaz. Doğru olan ise her kullanıcıya ayrı kimlik vermektir.

Yetki yükseltme (privilege escalation): Bir kullanıcının bazen normal seviyesinin üstünde bir şey çalıştırması gerekir. Linux'ta bu `sudo` veya `su` ile yapılır. Örnek: 1024'ün altındaki port numaraları korumalıdır. Web, FTP, DNS, SSH gibi bilinen servisler bu aralıkta çalışır. Sıradan bir kullanıcının programı bu portlardan birini açmaya çalışırsa hata alır.

Grup kimliği (group ID) sorusu. Öğrenci grup kimliğinin yönetici mi olduğunu sordu. Grup kimliğinin yöneticilikle ilgisi yoktur.

- UNIX'te her kullanıcının sayısal bir kullanıcı kimliği (UID) vardır. Örnek: 1001, 1005.
- 1000'in altındaki kimlikler genelde sistem servislerine ayrılmıştır.
- Bir kullanıcıya bir port açması için doğrudan root yetkisi vermek tehlikelidir. Root her şeyi yapabilir, kullanıcı başka bir şeyi de silebilir.
- Bunun yerine kullanıcı, belirli yetkileri olan bir gruba eklenir. Örnek: Dosyalarını web üzerinden yayınlayacak kullanıcı web servisinin grubuna eklenir.
- Linux'ta her kullanıcı için kendi adıyla aynı adlı bir grup da açılır.
- Bir dosyanın erişim hakları üç sınıf için ayrı tanımlanır: sahibi (user), grubu (group) ve diğerleri (others). Her biri için okuma, yazma ve çalıştırma (r, w, x) hakkı vardır.

Çekirdekteki veri yapıları

Veri yapıları dersinde görülen neredeyse her yapı çekirdekte kullanılır.

- **Bağlı liste ve çift yönlü bağlı liste:** Süreç denetim bloklarını (PCB) birbirine bağlamak için kullanılır. Süreç oluşunca listeye eklenir, bitince çıkarılır.
- **Dairesel liste:** G/Ç aygıtlarının bekleme kuyruğu gibi boyutu sabit yapılar için uygundur.
- **İkili arama ağacı.**
- **Hash:** Günlük kullanımda çok yaygındır. Bazı sistem servisleri hash mantığı üzerine kuruludur.

Süreçler de ağaç yapısında oluşur. Açılıştan sonra işletim sistemi kendi sürecinin bir kopyasını oluşturur ve bu kopyaya diskteki bir programı yükler. Kabuk (shell) da bir komutu böyle çalıştırır.

İşletim sistemi servisleri ve kullanıcı arayüzleri

İşletim sistemi programların ve servislerin çalışması için bir ortam sunar. Sunduğu servisler:

- **Kullanıcı arayüzü:**
  - Grafik arayüz (GUI).
  - Komut satırı arayüzü (CLI). Buna konsol veya kabuk (shell) da denir.
  - Toplu iş (batch) arayüzü. Komutlar art arda dizilip otomatik çalıştırılır.
- **Program çalıştırma:** Diskteki programı belleğe alıp komut komut çalıştırmak.
- **G/Ç işlemleri.**
- **Dosya sistemi işlemleri:** dosya oluşturma, silme, değiştirme ve erişim haklarını yönetme.
- **İletişim:** paylaşılan bellek (shared memory) veya mesaj aktarma (message passing).
  - Paylaşılan bellekte bir bellek alanı birden fazla süreç tarafından okunur ve yazılır.
  - Mesaj aktarma, bağımsız sistemler arasında da kullanılır. Kolaylıkları yüzünden tek bir sistem içinde de kullanılır.
- **Hata algılama:** Kullanıcı uygulamasının mantık hatası değil, yetkisiz komut çalıştırma veya izinsiz belleğe yazma gibi hatalar. İşletim sistemi bu durumda genelde uygulamayı durdurur.
- **Kaynak ayırma (resource allocation).**
- **Muhasebe (accounting):** Hangi kullanıcının hangi kaynağı ne kadar ve ne zaman kullandığının kaydı.
- **Koruma ve güvenlik:** Kayıt (log) tutma, ör. giriş denemeleri. Başarılı işlemlerin de kaydının tutulduğu denetim kayıtları (audit log), bir sorun anında geriye dönüp bakmayı sağlar.

Slayttaki blok şemada en altta donanım, üstünde işletim sistemi servisleri vardır. Bunların üstünde boydan boya **sistem çağrıları** katmanı vardır. Bu katman bir Çin Seddi gibi alttaki kaynaklara kontrolsüz erişimi engeller. En üstte kullanıcı arayüzleri bulunur. Bütün istekler sistem çağrıları üzerinden işletim sistemine geçer.

Komut satırı nasıl çalışır? Slayttaki eski bir macOS terminal görüntüsünde `ls`, `pwd` ve `ping` komutları vardır. Bunların çoğu kabuğun kendisi değil, ayrı programlardır. Kabuk şöyle çalışır:

1. Girilen komutu ayrıştırır (parse eder). Komutun var olup olmadığına, bazen parametrelerine bakar.
2. `fork` ile kendisinin bir kopyasını oluşturur.
3. Bu kopyaya `ping` programını yükler. Kopyanın kod alanı `ping`'in koduyla değiştirilir.
4. Program baştan çalışır, çıktısı ekrana gelir.

Komut satırı bir laboratuvar uygulamasında ayrıca işlenecek.

Grafik arayüz masaüstü benzetmesine (desktop metaphor) dayanır. Pencereli yapı Windows'tan önce de vardı, Windows onu yaygınlaştırdı. Çoğu sistem hem GUI hem CLI sunar. Tablet ve telefonlarda giriş klavye ve fare yerine dokunma ve hareketlerle yapılır.

Sistem çağrıları ve API

Sistem çağrıları, işletim sisteminin servislerine erişmeyi sağlar. Çoğu işletim sistemi C veya C++ ile yazıldığı için sistem çağrıları da bu seviyededir. Programlara sistem çağrıları doğrudan değil, bir **uygulama programlama arayüzü (API)** üzerinden sunulur.

- Windows'ta Win32 API kullanılır.
- POSIX tabanlı sistemlerde (UNIX, Linux, macOS) POSIX API kullanılır.

Örnek: `printf` bir fonksiyondur. Biçim dizgisi (format string) alır, sınırsız sayıda parametre alır, tip dönüşümlerini yapar ve sonunda `write` sistem çağrısıyla ekrana yazar. Aynı işi doğrudan `write` ile yapmaya çalışmak çok daha zahmetlidir. `man printf` ve `man write` ile ikisini karşılaştırabilirsiniz.

`read` sistem çağrısının API'si şöyledir:

```
ssize_t read(int fd, void *buf, size_t count);
```

- `fd`: dosya tanımlayıcısı (file descriptor). `FILE *` işaretçisi değildir.
- `buf`: okunan verinin konulacağı tampon.
- `count`: okunacak bayt sayısı.

Dosya kopyalama örneği. Slayttaki örnek, bir dosyanın içeriğini başka bir dosyaya kopyalarken ne kadar çok sistem çağrısı yapıldığını gösterir:

1. Kaynak dosya adını al: ekrana bir istem (prompt) yaz, kullanıcının girdisini oku.
2. Hedef dosya adını aynı yolla al.
3. Kaynak dosyayı aç. Dosya yoksa programı anormal bitir (abort).
4. Hedef dosyayı oluştur. Dosya zaten varsa programı anormal bitir.
5. Döngü: kaynaktan oku, hedefe yaz. Okuma başarısız olana (dosya sonuna gelene) kadar sür.
6. Hedef dosyayı kapat, ekrana tamamlandı mesajı yaz, normal bitir.

Döngüdeki her okuma ve yazma ayrı bir sistem çağrısıdır.

Sistem çağrısının işlenmesi. Kullanıcı uygulaması `open`'ı çağırır. Bir kesme oluşur ve çekirdek moduna geçilir. Her sistem çağrısının bir numarası vardır. Çağrılar bu numaralarla bir tabloda tutulur. Tablodan ilgili kod bulunup çalıştırılır. Sonuç kullanıcıya döndürülür. `read` gibi fonksiyonları sağlayan katman C çalışma zamanı kütüphanesidir.

Parametre aktarma yöntemleri. Yalnız çağrının numarası yetmez. Örnek: Yazma yaparken yazılacak yer, tamponun adresi ve boyut gerekir.

1. **Yazmaçlar:** En hızlı yoldur. Ama Intel mimarisinde yazmaç sayısı sınırlıdır ve bir kısmı belirli işlere ayrılmıştır. Çok parametre için uygun değildir.
2. **Bellekte blok (tablo):** Parametreler bellekte bir bloğa konur. Bloğun başlangıç adresi bir yazmaçla verilir. Kitaba göre Linux ve Solaris bunu kullanır.
3. **Yığıt (stack):** Parametreler yığıta itilir (push). Çekirdek tarafı bunları çeker (pop).

Blok ve yığıt yöntemlerinde parametre sayısı ve boyutu sınırlanmaz.

Sistem çağrısı türleri

Yüzlerce sistem çağrısı vardır. Başlıca gruplar:

- **Süreç denetimi:** oluşturma, sonlandırma, anormal sonlandırma (abort), öznitelik alma ve değiştirme, bir olayı bekleme, sinyal gönderme ve alma, bellek dökümü (dump), hata ayıklama.
- **Dosya yönetimi:** oluşturma, silme, açma, kapama, okuma, yazma, öznitelik alma ve değiştirme.
- **Aygıt yönetimi:** dosya yönetimine benzer işlemler.
- **Bilgi bakımı:** saat ve tarih alma ve ayarlama, sistem verisi alma ve ayarlama.
- **İletişim:** mesaj gönderme ve alma, paylaşılan bellek alanı oluşturma ve ona erişim hakkı alma.
- **Koruma:** kaynaklara erişim izinlerini verme ve kaldırma.

İşlem	Windows	UNIX
Süreç oluşturma	`CreateProcess()`	`fork()`
Süreçten çıkma	`ExitProcess()`	`exit()`
Bekleme	`WaitForSingleObject()`	`wait()`
Dosya oluşturma ve açma	`CreateFile()`	`open()`

POSIX uyumlu sistemlerde sistem çağrıları aşağı yukarı aynı adlara sahiptir.

Boru (pipe) sorusu

Öğrenci borunun paylaşılan bellek örneği olup olmadığını sordu. Boru paylaşılan bellek değildir.

- Bir süreç `fork` ile kendisinin bir kopyasını oluşturabilir. Bu iki süreç arasında ana-çocuk (parent-child) ilişkisi vardır.
- Çocuk süreç, ana sürecin açtığı kaynaklara aynı erişim hakkıyla sahip olur. Örnek: Açık bir dosyanın tanımlayıcısı çocukta da vardır.
- Boru, bu tür ilişkili süreçler arasında iletişim için çekirdeğin sunduğu bir yapıdır.
- Boru tek yönlüdür (simplex). Bir ucu yazmaya, diğer ucu okumaya açıktır.
- Önce boru oluşturulur, sonra `fork` yapılır. Bir süreç yazma ucunu, diğeri okuma ucunu kullanır.
- İki yönlü iletişim için iki boru oluşturulur ve ters yönlerde kullanılır.

```c
int fd[2];
pipe(fd);            /* fd[0] okuma ucu, fd[1] yazma ucu */
if (fork() == 0) {   /* çocuk süreç */
    close(fd[1]);
    char buf[32];
    read(fd[0], buf, sizeof buf);
} else {             /* ana süreç */
    close(fd[0]);
    write(fd[1], "merhaba", 8);
}
```

Laboratuvar duyurusu

Laboratuvar planlaması bir iki hafta içinde yapılacak. Uygulamaların 5. veya 6. haftadan itibaren başlaması bekleniyor. Asistanlar hazırlıklarını bitirince bir duyuru yapılacak.

Ders sonrası notlar

- **Dosya kopyalama slaytı:** Hoca, slaytta “Acquire input file name” satırının “Write prompt to screen” satırından önce gelmesini bir sıra hatası olarak yorumladı. Slayt doğrudur. Slayt karesinde “Acquire input file name” bir başlıktır. Altındaki girintili iki satır (“Write prompt to screen”, “Accept input”) bu adımı gerçekleştiren sistem çağrılarıdır. Yani önce istem yazılır, sonra girdi okunur. Hocanın anlattığı sıra da budur.
- **Linux’ta parametre aktarma:** Kitap, Linux ve Solaris’in blok yöntemini kullandığını söyler. Linux’un x86-64 sürümünde sistem çağrısı parametreleri (en fazla 6 tane) büyük olasılıkla doğrudan yazmaçlarla aktarılır. Kitaptaki bilgi daha çok genel bir örnektir.
- **Korumalı portlar:** Derste “1024 ve altı” dendi. Ayrıcalıklı portlar 0 ile 1023 arasıdır. 1024 numaralı port sıradan kullanıcıya açıktır.
- **UID sınırı:** 1000’in altındaki kimliklerin sisteme ayrılması dağıtıma göre değişir. Debian ve Ubuntu’da sıradan kullanıcılar 1000’den başlar. Eski Red Hat sürümlerinde sınır büyük olasılıkla 500’dü.
- **`read` ve dosya tanımlayıcısı:** Hocanın da vurguladığı gibi `read` bir `FILE *` değil, tam sayı bir dosya tanımlayıcısı alır. `FILE *` alan karşılık fonksiyon `fread`’dir.
- **Altyazı hataları:** “interroft/interhub/InterRap” interrupt, “COSIX” POSIX, “OSX tarafından” POSIX standardı, “Ayo” G/Ç (I/O), “Pink” ping demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.